

BADDEL LAUNCHER

How every launcher works · How playtime is tracked · Where data lives
v1.0 — March 2026

6 Platforms

Playtime Tracker

Live IPC

Win10 Fixed

1. Platform Support Overview

Platform	Scanner Source	Launch Command Format	Path Source
Steam	steamapps/*.acf manifests via libraryfolders.vdf	<code>steam://run/{appId}</code>	steamapps/common/{dir}
Epic Games	ProgramData/Epic/.../Manifests/*.item JSON	<code>com.epicgames.launcher://apps/{ns}%3A{id}%3A{app}?action=launch</code>	InstallLocation from manifest
Riot Games	ProgramData/Riot Games/RiotClientInstalls.json + folder scan	<code>"RiotClientServices.exe" --launch-product={valorant league_of_legends}</code>	Riot Games/{GameName} folder
Ubisoft Connect	HKLM Registry: Uninstall + Ubisoft\Launcher\Installs	<code>uplay://launch/{id}/0</code>	InstallLocation from registry
EA App	ProgramData/EA Desktop/InstallData + registry for paths	<code>origin2://game/launch?offerIds={offerId}</code>	HKLM Uninstall (Electronic Arts publisher)
Xbox / Store	Get-AppxPackage (PowerShell) — Store-signed, non-framework apps	<code>explorer.exe shell:AppsFolder\{PackageFamilyName}!App</code>	InstallLocation from Get-AppxPackage
Manual	User picks an .exe / .lnk file via file picker	<code>"C:\path\to\game.exe"</code>	Directory of chosen EXE

2. Platform Launch Deep-Dives

2.1 Steam

Registry key	HKCU\Software\Valve\Steam → SteamPath
Manifest path	{steamPath}\steamapps\libraryfolders.vdf + appmanifest_*.acf
Game ID format	steam-{appId} e.g. steam-730
Launch URI	steam://run/{appId}
Update flag	StateFlags field in .acf — "4" = fully installed

2.2 Epic Games

Manifest dir	%ProgramData%\Epic\EpicGamesLauncher\Data\Manifests\
Game ID format	epic-{AppName}
Launch URI	com.epicgames.launcher://apps/{ns}%3A{itemId}%3A{app}?action=launch&silent=true

2.3 Riot Games (Win10 fix included)

Windows 10 Issue & Fix: On Win10, ps-list sometimes cannot retrieve the full command-line arguments of child processes spawned by RiotClientServices.exe. This means the --launch-product flag is invisible to the process checker, so games appear to never start. The fix adds a direct EXE match list — **valorant-win64-shipping.exe, leagueclient.exe, league of legends.exe** — that is checked before the command-line approach. If any of these processes exist, the game is immediately considered running regardless of command-line visibility.

Installs JSON	%ProgramData%\Riot Games\RiotClientInstalls.json
Client EXE	{riotBase}\Riot Client\RiotClientServices.exe
Launch cmd	"RiotClientServices.exe" --launch-product=valorant --launch-patchline=live

Win10 EXE match	valorant-win64-shipping.exe leagueclient.exe league of legends.exe
Game ID	Stable MD5 hash of command string (no numeric ID from Riot API)

2.4 Ubisoft Connect

Registry key 1	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\Uplay Install *
Registry key 2	HKLM\SOFTWARE\WOW6432Node\Ubisoft\Launcher\Installs\{id} → InstallDir
Launch URI	uplay://launch/{id}/0

2.5 EA App

Manifest dir	%ProgramData%\EA Desktop\InstallData\{gameFolder}\
Offer ID	base-{offerId} filename or JSON offerId / contentIds[0]
Path source	HKLM Uninstall where Publisher contains "Electronic Arts"
Launch URI	origin2://game/launch?offerIds={offerId}
Game ID format	ea-{offerId}

2.6 Xbox / Microsoft Store

Source	Get-AppxPackage (PowerShell) — Store-signed, IsFramework=false
Validation	SteamGridDB cover image lookup — no cover = excluded
Launch URI	explorer.exe shell:AppsFolder\{PackageFamilyName}!App
Bloat filter	~80 keyword blocklist + client/overlay/service name filter

Concurrency	3 parallel SteamGridDB lookups (CONCURRENCY = 3)
-------------	--

2.7 Manual Games

Input	.exe or .lnk (shortcut is resolved to target EXE)
Name search order	1. EXE filename → 2. User-typed name → 3. Parent folder name
Duplicate check	Existing game with same normalized command path → restore if hidden, skip if visible
Platform	"Manual"
ID generation	MD5 hash of normalized command string (first 16 hex chars)

3. Launch Flow (main.js)

Step	Condition	Action
1	command contains "://"	exec("start \"\" + cleanCmd + "\"") — uses Windows URI protocol handler. Falls back to shell.openExternal() if exec fails.
2	command ends with ".lnk"	shell.openPath(cleanCmd) — opens shortcut via Windows shell
3	all other commands	exec(command) directly — waits 500 ms; if no error → success
4	launchSuccess === true	startGameTracking(gameId, command, gamePath, gameName) called immediately
5	always	analytics.logGameLaunched(platform) fired (consent-gated)

4. Process Detection — isGameRunning()

Priority	Method	Details
1 — Dynamic EXE	Read game folder for .exe files	getDynamicGameExes() scans game path + common subdirs (Bin, Binaries, Win64, Win32, Core, Retail). Cached after first read. Filters out uninstall/crash/setup/anticheat EXEs.
2 — Riot EXE list	Direct EXE name match (Win10 fix)	Checks against: valorant-win64-shipping.exe, leagueclient.exe, league of legends.exe — no command-line parsing needed. Works on Win10 where cmd args may be hidden.
3 — Riot cmd args	--launch-product flag in command line	If command includes riotclientservices.exe, extracts --launch-product and maps to target EXE. Win11 primary path.
4 — Path match	Process cmd contains game path	pCmd.includes(pathLower) — reliable for EXEs launched directly (Manual games, some Steam/Epic games)
5 — Name fuzzy match	Cleaned process name vs game name	Strips non-alphanumeric chars. Checks equality, substring match, and dynamic aliases (e.g. "acvalhalla" for "Assassins Creed Valhalla")

5. Playtime Tracker — Full Lifecycle

5.1 Tracker State Machine

State	Trigger	Variables set
CREATED	startGameTracking() called on launch	clickTime = Date.now() • wasRunning = false • checkCount = 0 • lastSavedMinutes = 0
WAITING	isGameRunning() returns false, < 5 min since click	Checks continue every 10 s. Allows slow-launching games (anti-cheat, launchers) up to 5 minutes.
RUNNING	isGameRunning() returns true for first time	actualStartTime = Date.now() • wasRunning = true — timer starts from this moment
PERIODIC SAVE	Every 5 minutes while RUNNING	updatePlaytime(gameId, 5) called • lastSavedMinutes += 5 — crash-safe incremental save
CLOSED	isGameRunning() → false after wasRunning = true	saveTrackerPlaytime() called with remaining unsaved minutes • tracker deleted
TIMEOUT	5 minutes since click, game never started	Tracker silently deleted. No playtime recorded.

5.2 Timing Accuracy

Tick interval	10 seconds (setInterval 10,000 ms)
Timer start	actualStartTime — set when game process first detected, NOT when Play was clicked
Periodic save	Every 5 accumulated minutes of actual game time

Final save	totalMinutes – lastSavedMinutes (only unsaved remainder is written)
Minimum session	0 minutes (sub-minute sessions are recorded as 0 and ignored at display)
Max startup wait	300,000 ms (5 minutes) before tracker gives up

5.3 Global Watcher (External Launch Detection)

6. Playtime Persistence

6.1 Data Schema per Game

Field	Type	Description
<code>totalPlaytime</code>	number (minutes)	Cumulative total across all sessions. Incremented by <code>updatePlaytime()</code> . Never reset.
<code>lastPlayed</code>	number (Unix ms)	Timestamp of most recent playtime write. Set to <code>Date.now()</code> on every <code>updatePlaytime()</code> call.
<code>playSessions</code>	Array of { date, minutes }	One entry per calendar day. Date format: YYYY-MM-DD (ISO date). If a session already exists for today, minutes are added to it.

6.2 `updatePlaytime()` Logic

Called from	<code>saveTrackerPlaytime()</code> in <code>main.js</code> (session end) + periodic 5-min save while game runs
Accumulation	<code>game.totalPlaytime += playedMinutes</code> (additive, never overwrites)
Daily bucket	<code>new Date().toISOString().split("T")[0]</code> → find or create <code>playSessions</code> entry for today
DB write	<code>saveDatabase()</code> with 500 ms debounce — rapid successive calls collapse into one write
Return value	{ status, totalPlaytime, lastPlayed, playSessions } — used by IPC to push live update to renderer

6.3 DB File

Path	<code>%APPDATA%\{app}\games-db.json</code>
Format	JSON array — one object per game, human-readable (2-space indent)
Backup	<code>games-db-backup.json</code> written automatically if primary save fails

Write guard	500 ms debounce timer (this._saveTimer) — prevents write storms from rapid updates
Corruption recovery	JSON.parse failure → dbCache reset to [] and immediately saved clean

7. Live Frontend Updates (IPC)

Channel	Direction	When	Payload
<code>update-playtime</code> (<code>ipcMain.handle</code>)	Renderer → Main → Renderer	Renderer calls <code>electronAPI.updatePlaytime(gameId, minutes)</code> manually	<code>{ status, totalPlaytime, lastPlayed, playSessions }</code>
<code>playtime-updated</code> (<code>webContents.send</code>)	Main → Renderer (push)	<code>saveTrackerPlaytime()</code> resolves after session ends — automatic, no renderer request needed	<code>{ gameId, totalPlaytime, lastPlayed, playSessions }</code>
<code>onPlaytimeUpdated</code> (preload listener)	Preload bridge	<code>electronAPI.onPlaytimeUpdated(cb)</code> registers a callback for the push event	Same payload as <code>playtime-updated</code> above

8. Game ID Stability

Platform	ID Format	Stability Guarantee
Steam	<code>steam-{appId}</code>	Valve <code>appId</code> never changes
Epic Games	<code>epic-{AppName}</code>	Epic <code>AppName</code> is a slug, stable across reinstalls
EA App	<code>ea-{offerId}</code>	EA offer ID from manifest JSON
Riot / Ubisoft / Manual	<code>MD5(command)[0:16]</code>	Hash of normalised launch command. Stable as long as install path does not change.

9. Image Pipeline

Source	SteamGridDB API (<code>services/steamgriddb.js</code>)
Cache location	<code>%APPDATA%\{app}\image_cache\{type}_{gameId}.{ext}</code>
Types	cover • hero • logo

Write strategy	Write to {file}.tmp first → rename to final path after close (atomic, prevents corrupt partials)
Timeout	7 seconds per download — times out cleanly, resolves null
DB fields	image / defaultImage (cover) • heroImage / defaultHero • logo / defaultLogo
Fallback	refetchMissingImages() scans all games on startup and re-downloads missing artwork silently

This document covers gameScanner.js and the relevant sections of main.js as shipped in the March 2026 production build. The Win10 Riot fix (section 2.3 / 4) is included in this release.